

# Exploring Hybrid DFT– Quantum Embedding Simulations of Molecules for HPCQC

---

Presented By – Dr. Namrataa Kkommineni,  
Tejjan Arora,  
Soham Phulare,  
Shreyas Kadam,

Shah & Anchor Kutchhi Engineering College  
Guided By - Samrit Kumar Maity, Scientist E , HPC  
technologies, CDAC-Pune.

# **WORK COMPLETED BY**

## **TEJJAN ARORA**

# Exchange Functional PBE Convergence Test

Table 1: DFT Embedding with PBE Exchange: Energy Analysis

| Molecule         | Active Space | Reference DFT Energy (Ha) | Final Ground-State Energy (Ha) |
|------------------|--------------|---------------------------|--------------------------------|
| H <sub>2</sub>   | (2e, 2o)     | −1.161905                 | −1.182605                      |
| H <sub>2</sub> O | (2e, 6o)     | −76.320335                | −76.341891                     |
| H <sub>2</sub> O | (8e, 6o)     | −76.320335                | −76.353242                     |
| O <sub>2</sub>   | (2e, 6o)     | −150.174613               | −150.136870                    |
| O <sub>2</sub>   | (8e, 6o)     | −150.174613               | −150.300681                    |

Table 2: DFT Embedding with PBE Exchange: Convergence Analysis

| Molecule         | Active Space | Converged at Iteration | Energy Tolerance (Ha) | Wall Time(sec) |
|------------------|--------------|------------------------|-----------------------|----------------|
| H <sub>2</sub>   | (2e, 2o)     | 9                      | $7.669e-08$           | 61             |
| H <sub>2</sub> O | (2e, 6o)     | 9                      | $2.321e-07$           | 1081           |
| H <sub>2</sub> O | (8e, 6o)     | 9                      | $2.54e-07$            | 5,516          |
| O <sub>2</sub>   | (2e, 6o)     | 4                      | $3.599e-07$           | 141            |
| O <sub>2</sub>   | (8e, 6o)     | 27                     | $5.873e-07$           | 9217           |

# Exchange Functional B3LYP Convergence Test

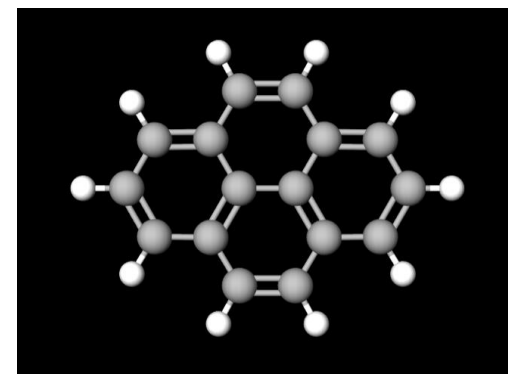
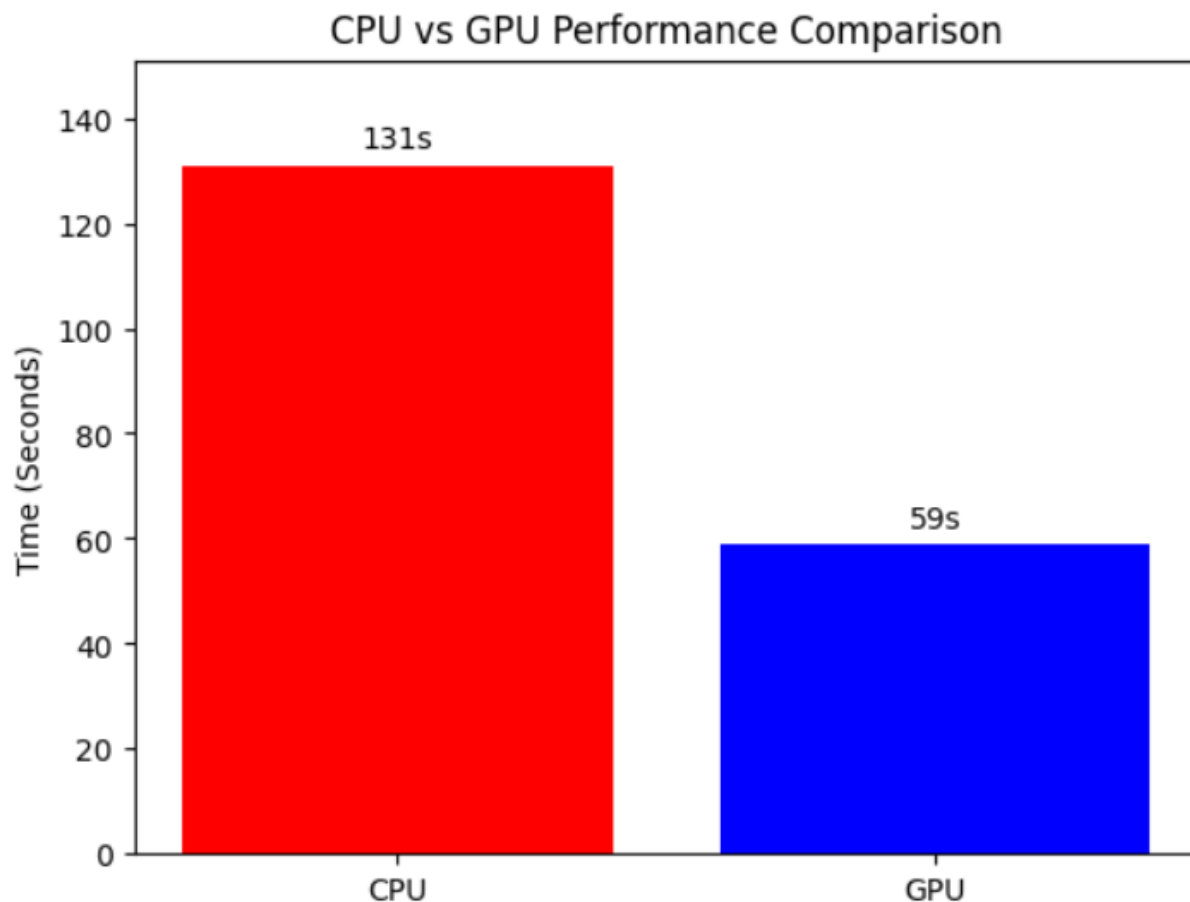
Table 3: B3LYP Exchange: Energy Analysis ( $\alpha_{\text{init}} = 0.6$ )

| Molecule         | Active Space | Reference DFT Energy (Ha) | Final Ground-State Energy (Ha) |
|------------------|--------------|---------------------------|--------------------------------|
| H <sub>2</sub>   | (2e, 2o)     | -1.175477                 | -1.190061                      |
| H <sub>2</sub> O | (2e, 6o)     | -76.407023                | -76.425277                     |
| H <sub>2</sub> O | (8e, 6o)     | -76.407023                | -76.431931                     |
| O <sub>2</sub>   | (2e, 6o)     | -150.312682               | -150.220130                    |
| O <sub>2</sub>   | (8e, 6o)     | -150.312682               | -150.353251                    |

Table 4: B3LYP Exchange: Convergence Analysis ( $\alpha_{\text{init}} = 0.6$ )

| Molecule         | Active Space | Converged at Iteration | Energy Tolerance (Ha) | Wall Time(sec) |
|------------------|--------------|------------------------|-----------------------|----------------|
| H <sub>2</sub>   | (2e, 2o)     | 9                      | $6.683e-07$           | 61             |
| H <sub>2</sub> O | (2e, 6o)     | 9                      | $9.587e-07$           | 1115           |
| H <sub>2</sub> O | (8e, 6o)     | 10                     | $7.01e-07$            | 6,052          |
| O <sub>2</sub>   | (2e, 6o)     | 3                      | $7.153e-07$           | 123            |
| O <sub>2</sub>   | (8e, 6o)     | 14                     | $5.23e-07$            | 4,727          |

# CPU vs GPU Comparison Plot (Pyrene Molecule)



Pyrene ( $C_{16}H_{10}$ )

This comparison is justified by the fact that the GPU and CPU calculations produced identical energies (i.e.  $-615.760309$  Hartree).

Core libraries such as **PySCF** and **Qiskit** use **C/Rust backends designed for CPU RAM (NumPy arrays)**.

GPU arrays (**CuPy**) store data in **VRAM**, which these backends cannot directly access.

Python safety layers intercept this mismatch before a crash:

- **PySCF:** CuPy arrays trigger validation errors (e.g., *AssertionError* during density matrix operations).
- **Qiskit:** CuPy blocks automatic device-to-host conversions, causing a **TypeError**.

## Dynamic Namespace Injection

- Used Python's **sys.modules** to intercept all **NumPy imports**.
- Redirected them to the **CuPy GPU library** during runtime initialization.
- This forced the program to **use GPU arrays instead of CPU arrays** without modifying the original source code.

## API Differences Between NumPy and CuPy

- **Missing administrative modules:** Some NumPy utilities (e.g., `numpy.lib.NumpyVersion`) are not implemented in CuPy.
- **Memory immutability:** Qiskit uses NumPy's `.setflags(write=False)` to protect quantum gate definitions, which CuPy does not support.
- **Type conversion behavior:**
  - NumPy automatically converts **Python lists/tuples** → **arrays** during matrix operations.
  - CuPy requires **explicit conversion**, otherwise it raises errors.

## Runtime Attribute Mocking (Monkey Patching)

### Bypassing Version Checks

- Created a **mock version of `numpy.lib.NumpyVersion`**.
- This allowed **Qiskit initialization** to proceed even though NumPy was replaced with CuPy.

### Bypassing Memory Locks

- Added a **dummy `.setflags()` method** to the `cupy.ndarray` class.
- This bypassed Qiskit's **read-only memory protection** used for quantum gate definitions.

## Hardware Latency (PCIe Bottleneck)

- Data must move between **CPU RAM and GPU VRAM through the PCIe bus**.
- For **small systems** (e.g., active space: **6 orbitals, 2 electrons**), **data transfer time exceeds the GPU compute benefit**.

## Diagnostic Outcome

- These runtime patches **bypassed Python-level compatibility checks**.
- This allowed us to **identify the exact boundaries** where:
  - **C-based PySCF components**, and
  - **Qiskit's Rust backend** strictly require **CPU memory (NumPy arrays)**.

# Thank You!!

? Questions